

Python Reporting with Pagila DWH

Vibe Coding with AI Assistance

Data Management Analysis and Reporting (DMAR)
Winter Term 2025/26

Overview

In this exercise, you will create a simple reporting application using the Pagila Data Warehouse that you set up in the PostgreSQL exercise. The goal is to generate two meaningful reports with visualizations.

Vibe Coding – AI-Assisted Development

This exercise is designed for **AI-assisted programming** (“Vibe Coding”). You are **encouraged** to use AI tools such as:

- Claude, ChatGPT, or similar LLMs
- GitHub Copilot
- Cursor IDE

Your task: Describe what you want to build, let the AI generate the code, then test and refine it. Document your prompts and learnings!

Prerequisites

- PostgreSQL installed with the **Pagila DWH** database (from Exercise 02)
- Python 3.x installed
- Basic understanding of SQL (SELECT, JOIN, GROUP BY)

Resources

- **Reader E: Python Short Course** – available on Moodle
- **Pagila DWH Schema:** See Exercise 03 materials

1 The Pagila DWH Schema

The Pagila Data Warehouse contains rental data from a DVD rental business:

Table	Description
dim_date	Time dimension (day, month, quarter, year)
dim_film	Films (title, category, rating, length)
dim_customer	Customers (name, city, country)
dim_store	Stores (manager, location)
fact_rental	Rental transactions (amount, days, fees)
vw_rental_analysis	Denormalized view for easy queries

Tip: Use the View

The view `vw_rental_analysis` already joins all tables – this makes your SQL queries much simpler!

2 Your Task: Create Two Reports

Create a Python application that generates **two reports** from the Pagila DWH:

Report 1: Rentals by Film Category (Bar Chart)

Create a bar chart showing:

- X-axis: Film categories (Action, Comedy, Drama, etc.)
- Y-axis: Total number of rentals OR total revenue

Example SQL Query:

```
SELECT film_category,
       COUNT(*) AS total_rentals,
       SUM(rental_amount) AS total_revenue
FROM vw_rental_analysis
GROUP BY film_category
ORDER BY total_rentals DESC;
```

Report 2: Rental Trends Over Time (Line Chart)

Create a line chart showing:

- X-axis: Time (months or quarters)
- Y-axis: Number of rentals or revenue

Example SQL Query:

```
SELECT year, month, month_name,  
       COUNT(*) AS total_rentals,  
       SUM(rental_amount) AS total_revenue  
FROM vw_rental_analysis  
GROUP BY year, month, month_name  
ORDER BY year, month;
```

3 Technology Options

Choose **one** of the following approaches:

Option	Tools	Best For
A: Streamlit	streamlit, pandas, plotly	Interactive web dashboard
B: Jupyter	jupyter, pandas, matplotlib	Data exploration notebook
C: Script	pandas, matplotlib	Simple Python script with charts
D: Other	Power BI, Tableau, Excel	Non-Python alternatives

Recommended: Option A (Streamlit)

Streamlit is easy to use and creates interactive web apps with just a few lines of code. Perfect for AI-assisted development!

Installation:

```
pip install streamlit pandas psycopg2-binary plotly
```

Run your app:

```
streamlit run your_app.py
```

4 Setup Steps

Follow these steps to set up your Python environment:

Step 1: Install Python

Install Python 3.10 or higher from <https://www.python.org/downloads/>

Verify the installation:

```
python --version
```

Step 2: Install Required Packages

Install the PostgreSQL adapter and other dependencies:

```
pip install psycopg2-binary pandas
```

For Streamlit (recommended):

```
pip install streamlit plotly
```

Step 3: Test Database Connection

Create a test script to verify the connection to your Pagila database:

```
import psycopg2

conn = psycopg2.connect (
    host="localhost",
    database="pagila",
    user="your_username",
    password="your_password"
)

cursor = conn.cursor()
cursor.execute("SELECT COUNT(*) FROM vw_rental_analysis")
print(f"Total records: {cursor.fetchone()[0]}")

conn.close()
```

If you see a count (e.g., “Total records: 16044”), your connection is working!

5 Getting Started with AI

Step 1: Describe Your Goal

Tell the AI what you want to build. Example prompt:

“I want to create a Streamlit app that connects to a PostgreSQL database called 'pagila' and shows two charts: (1) a bar chart of rentals by film category, and (2) a line chart of monthly rental trends. The database has a view called vw_rental_analysis with columns: film_category, year, month, month_name, rental_amount.”

Step 2: Test and Refine

1. Run the generated code
2. If errors occur, copy the error message to the AI
3. Ask for fixes or improvements
4. Repeat until it works

Step 3: Customize

Once the basics work, ask the AI to:

- Add filters (by year, category, etc.)
- Improve the chart styling
- Add summary statistics

6 Submission

Submit the following via Moodle:

1. A link to your code repository (GitHub, GitLab, or similar)
2. A `README.md` in your repository describing:
 - Which AI tool you used
 - Your main prompts
 - What worked well / what was challenging
3. Screenshots of your two reports (in the repository or uploaded separately)

7 Learning Goals

After completing this exercise, you should be able to:

- Connect Python to PostgreSQL
- Execute SQL queries and load results into pandas
- Create basic visualizations (bar charts, line charts)
- Use AI tools effectively for coding tasks
- Build a simple reporting application